

OneMasterFramework

Actual Architecture & Framework Structure

TEST WebBase / Test	CONFIGURATION FrameworkConfig / BrowserStackConfig	FACTORY Select execution strategy
MANAGER Browser-specific setup	DRIVER WebDriver / Mobile Driver	EXECUTION Local / Remote / Grid

1. Actual Project Structure

The following structure is based on the IntelliJ project screenshots you provided. It is the actual visible package/file structure, not a generic proposed framework layout.

```

OneMasterFramework/
  .idea/
  src/
    main/
      java/
        com.tmb/
          config/
            converters/
              factory/
                BrowserStackConfigFactory
                ConfigFactory
                StringToBrowserTypeConverter
                StringToRemoteModeBrowserConverter
                StringToRunModeBrowserTypeConverter
                StringToURLConverter
            BrowserStackConfig
            FrameworkConfig
          driver/
            factory/
              mobile/
                local/LocalMobileDriverFactory
                remote/
                  BrowserStackMobileFactory
                  RemoteMobileDriverFactory
              web/
                local/LocalDriverFactory
                remote/
                  BrowserStackFactory
                  RemoteDriverFactory
                  SauceLabFactory
                  SeleniumGridFactory
                  SelenoidFactory
            Driver
          manager.web/
            local/
              ChromeManager
              FirefoxManager
            remote/
              browserstack/
                BrowserStackChromeManager
                BrowserStackFirefoxManager
              selenium/
                SeleniumGridChromeManager
                SeleniumGridFirefoxManager
              selenoid/
                SelenoidChromeManager
                SelenoidFirefoxManager
            enums/
              BrowserRemoteModeType
              BrowserType
              MobilePlatformType
              MobileRemoteModeType
              RunModeBrowserType
          utils/
            CommonMethods
          test/
            java/
              com.tmb.tests/
                DemoTest
                WebBase
            resources/
              browserstack-config.properties
              framework-config.properties
          target/
            FrameworkInfo.yaml
          pom.xml
          README.md

```

2. Configuration Layer

config/ centralises framework configuration. The visible structure contains FrameworkConfig and BrowserStackConfig plus converters and factories.

Key responsibilities

- Read framework properties and expose typed configuration.
- Convert string configuration values into enum/URL types.
- Keep BrowserStack-specific configuration separate from general framework configuration.

Important classes

FrameworkConfig — main framework configuration interface/type.

BrowserStackConfig — BrowserStack-specific configuration.

ConfigFactory — configuration creation/access layer.

BrowserStackConfigFactory — BrowserStack configuration factory.

Converters — transform string property values into framework-specific types.

3. Driver Factory Layer

The driver/factory package separates driver creation by platform and execution mode.

Area	Actual implementation visible
Web / Local	LocalDriverFactory
Web / Remote	BrowserStackFactory, RemoteDriverFactory, SauceLabFactory, SeleniumGridFactory, SelenoidFactory
Mobile / Local	LocalMobileDriverFactory
Mobile / Remote	BrowserStackMobileFactory, RemoteMobileDriverFactory

4. Browser Manager Layer

manager.web separates browser-specific setup from the higher-level factory. The visible implementation supports local Chrome/Firefox and remote BrowserStack, Selenium Grid and Selenoid managers.

Execution	Managers
Local	ChromeManager, FirefoxManager
BrowserStack	BrowserStackChromeManager, BrowserStackFirefoxManager
Selenium Grid	SeleniumGridChromeManager, SeleniumGridFirefoxManager
Selenoid	SelenoidChromeManager, SelenoidFirefoxManager

5. Enums — Strongly Typed Framework Options

The enums package contains `BrowserRemoteModeType`, `BrowserType`, `MobilePlatformType`, `MobileRemoteModeType` and `RunModeBrowserType`. These provide controlled values for browser, platform and execution-mode decisions.

6. Test Layer

The test source tree contains **com.tmb.tests** with `DemoTest` and `WebBase`. The Maven convention separates reusable framework implementation under `src/main/java` from tests under `src/test/java`.

7. Resources

`src/test/resources` contains **framework-config.properties** and **browserstack-config.properties**. These are the visible external configuration files associated with the configuration layer.

8. Actual Execution Concept

```
graph TD;
    Test --> WebBase[Test / WebBase];
    WebBase --> Configuration;
    Configuration --> FrameworkConfig[FrameworkConfig / BrowserStackConfig];
    FrameworkConfig --> Enum[Enum + Converter];
    Enum --> DriverFactory[Driver Factory];
    DriverFactory --> Execution[Local / BrowserStack / Selenium Grid / Selenoid / Sauce Labs];
    Execution --> BrowserManager[Browser Manager];
    BrowserManager --> WebDriver[WebDriver / Mobile Driver];
    WebDriver --> TestExecution[Test execution];
```

9. Senior SDET Interview Explanation

“OneMasterFramework is a Java-based automation framework where I separated configuration, driver factories and browser managers. The configuration layer converts external property values into strongly typed framework options. The factory layer selects the execution strategy, such as local, BrowserStack, Selenium Grid, Selenoid or Sauce Labs, while the manager layer handles browser-specific setup. This separation keeps tests independent of the execution environment and makes the framework easier to extend.”

10. Key Design Principle

Test should not know how the driver is created.

The test requests the required execution setup; configuration, factories and managers handle the implementation details. This is the central architectural idea visible in your project structure.